Z1 · 4-bitowy układ kombinacyjny

UCiSW 1 · Laboratorium 1 · Vivado + płyta AUP-ZU3

WE **X(3:0)**, WY **Y(3:0)** — wektory 4-bitowe w NKB (0–15)

a) Y = (X + 1) mod 16 | **b) Y = (X + 3) mod 16** | **c) Y = (X – 1) mod 16** | **d) Y = (X – 3) mod 16**

Co trzeba zrobić

1. Minimalizacja Y(0)…Y(3) w tablicach Karnaugha → równania
2. Opis VHDL (równania logiczne)
3. Symulacja behawioralna: wszystkie 16 kombinacji X (krok 100 ns)
4. Sprzęt: X → PL_USER_SW(3:0), Y → PL_USER_LED(3:0)
5. Symulacja czasowa: przełączanie WY przy X: 0→1

Pliki w folderze Z1/

- `Z1_a.vhd` … `Z1_d.vhd` — encja `Z1` (dodaj **tylko swój** wariant)
- `Z1_tb.vhd` — testbench + samosprawdzenie (generic `K`)
- `Z1.xdc` — piny SW → X, LED → Y

Ściąga: równania wszystkich wariantów

Wariant	Y(3)	Y(2)	Y(1)	Y(0)
a) +1	$x_3\bar{x_2} + x_3\bar{x_1} + x_3\bar{x_0} + \bar{x_3}x_2x_1x_0$ = $x_3 \oplus (x_2x_1x_0)$	$x_2\bar{x_1} + x_2\bar{x_0} + \bar{x_2}x_1x_0$ = $x_2 \oplus (x_1x_0)$	$x_1\bar{x_0} + \bar{x_1}x_0$ = $x_1 \oplus x_0$	$\bar{x_0}$
b) +3	$x_3\bar{x_2} + x_3\bar{x_1}\bar{x_0} + \bar{x_3}x_2x_1 + \bar{x_3}x_2x_0$ = $x_3 \oplus x_2(x_1 + x_0)$	$x_2\bar{x_1}\bar{x_0} + \bar{x_2}x_1 + \bar{x_2}x_0$ = $x_2 \oplus (x_1 + x_0)$	$x_1x_0 + \bar{x_1}\bar{x_0}$ = $x_1 \odot x_0$	$\bar{x_0}$
c) –1	$x_3x_2 + x_3x_1 + x_3x_0 + \bar{x_3}\bar{x_2}\bar{x_1}\bar{x_0}$ = $x_3 \oplus (\bar{x}_2\bar{x}_1\bar{x}_0)$	$x_2x_1 + x_2x_0 + \bar{x_2}\bar{x_1}\bar{x_0}$ = $x_2 \oplus (\bar{x}_1\bar{x}_0)$	$x_1x_0 + \bar{x_1}\bar{x_0}$ = $x_1 \odot x_0$	$\bar{x_0}$
d) –3	$x_3x_2 + x_3x_1x_0 + \bar{x_3}\bar{x_2}\bar{x_1} + \bar{x_3}\bar{x_2}\bar{x_0}$ = $x_3 \oplus \bar{x}_2(\bar{x}_1 + \bar{x}_0)$	$x_2x_1x_0 + \bar{x_2}\bar{x_1} + \bar{x_2}\bar{x_0}$ = $x_2 \odot (x_1x_0)$	$x_1\bar{x_0} + \bar{x_1}x_0$ = $x_1 \oplus x_0$	$\bar{x_0}$

Notacja: kreska nad zmienną = NOT, sąsiadowanie = AND, + = OR, ⊕ = XOR, ⊙ = XNOR. Wszystkie równania sprawdzone dla 16 wartości X (skrypt + symulacja VHDL w nvc).

Tablica prawdy

X	X (bin)	a) X+1	b) X+3	c) X−1	d) X−3
0	0000	0001 (1)	0011 (3)	1111 (15)	1101 (13)
1	0001	0010 (2)	0100 (4)	0000 (0)	1110 (14)
2	0010	0011 (3)	0101 (5)	0001 (1)	1111 (15)
3	0011	0100 (4)	0110 (6)	0010 (2)	0000 (0)
4	0100	0101 (5)	0111 (7)	0011 (3)	0001 (1)
5	0101	0110 (6)	1000 (8)	0100 (4)	0010 (2)
6	0110	0111 (7)	1001 (9)	0101 (5)	0011 (3)
7	0111	1000 (8)	1010 (10)	0110 (6)	0100 (4)
8	1000	1001 (9)	1011 (11)	0111 (7)	0101 (5)
9	1001	1010 (10)	1100 (12)	1000 (8)	0110 (6)
10	1010	1011 (11)	1101 (13)	1001 (9)	0111 (7)
11	1011	1100 (12)	1110 (14)	1010 (10)	1000 (8)
12	1100	1101 (13)	1111 (15)	1011 (11)	1001 (9)
13	1101	1110 (14)	0000 (0)	1100 (12)	1010 (10)
14	1110	1111 (15)	0001 (1)	1101 (13)	1011 (11)
15	1111	0000 (0)	0010 (2)	1110 (14)	1100 (12)

Jak czytać tablice Karnaugha na kolejnych stronach

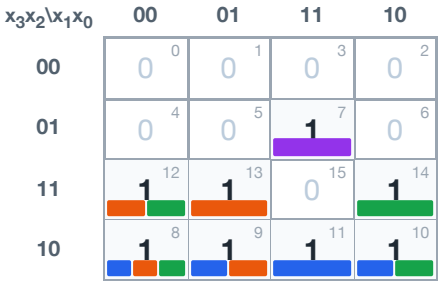
Wiersze: x_3x_2, kolumny: x_1x_0, oba w kodzie Graya (00, 01, 11, 10). Mała szara liczba w rogu komórki = wartość X. Kolorowe paski u dołu komórki = grupy (implikanty), które ją pokrywają — kolor paska odpowiada kolorowi składnika w równaniu pod mapą. Skrajne kolumny i skrajne wiersze są sąsiednie (mapa „zawija się").

Spostrzeżenia do powiedzenia prowadzącemu

- **Y(0) = $\bar{x}_0$** zawsze — dodanie/odjęcie liczby nieparzystej odwraca najmłodszy bit.
- **Y(1)** to zawsze XOR lub XNOR — „szachownica" na mapie, nic się nie skleja w większe grupy.
- **c) jest lustrem a)**, **d) lustrem b)**: c(X) = NOT a(NOT X), bo 15 − ((15 − X) + 1) = X − 1.
- W FPGA każde Y(i) zależy od ≤ 4 wejść → mieści się w jednej **LUT6**; minimalizacja ma sens dydaktyczny (i dla układów z bramek), Vivado i tak optymalizuje sam.

Wariant a) Y = (X + 1) mod 16

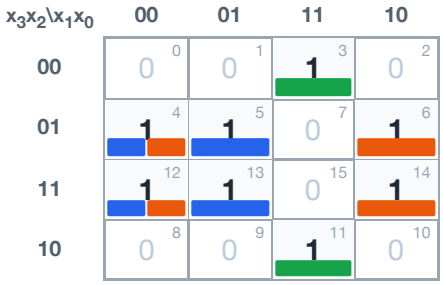
Y(3)



$Y(3) = x_3\overline{x_2} + \overline{x_3}x_1 + x_3\overline{x_0} + \overline{x_3}x_2x_1x_0$

$= x_3 \oplus (x_2x_1x_0)$

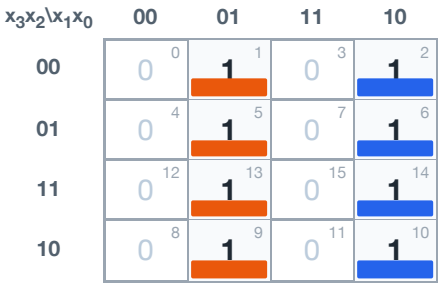
Y(2)



$Y(2) = x_2\overline{x_1} + \overline{x_2}x_0 + \overline{x_2}x_1x_0$

$= x_2 \oplus (x_1x_0)$

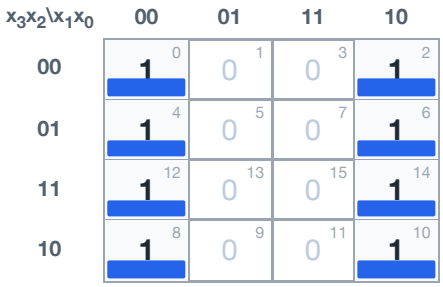
Y(1)



$Y(1) = x_1\overline{x_0} + \overline{x_1}x_0$

$= x_1 \oplus x_0$

Y(0)



$Y(0) = \overline{x_0}$

$= \bar{x}_0$

> **Intuicja**
>
> Bit i przełącza się, gdy wszystkie niższe bity = 1 (przeniesienie).

Kod VHDL — `Z1_a.vhd`

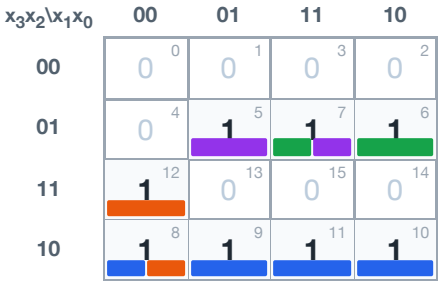
```
-- Z1 a) Y = (X + 1) mod 16
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Z1 is
    Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
           Y : out STD_LOGIC_VECTOR (3 downto 0));
end Z1;

architecture Dataflow of Z1 is
begin
    Y(0) <= not X(0);
    Y(1) <= (X(1) and not X(0)) or (not X(1) and X(0));
    Y(2) <= (X(2) and not X(1)) or (X(2) and not X(0))
         or (not X(2) and X(1) and X(0));
    Y(3) <= (X(3) and not X(2)) or (X(3) and not X(1)) or (X(3) and not X(0))
         or (not X(3) and X(2) and X(1) and X(0));
end Dataflow;
```

Wariant b) Y = (X + 3) mod 16

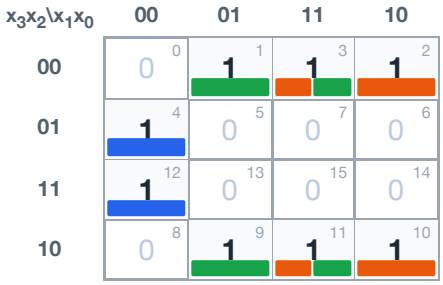
Y(3)



$Y(3) = x_3\bar{x_2} + x_3\bar{x_1}\bar{x_0} + \bar{x_3}x_2x_1 + \bar{x_3}x_2x_0$

$= x_3 \oplus x_2(x_1 + x_0)$

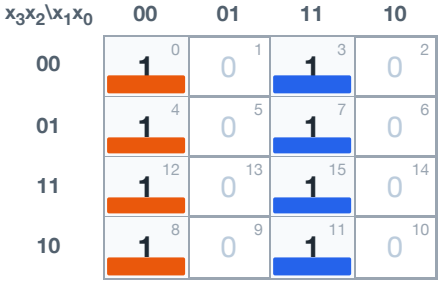
Y(2)



$Y(2) = x_2\bar{x_1}\bar{x_0} + \bar{x_2}x_1 + \bar{x_2}x_0$

$= x_2 \oplus (x_1 + x_0)$

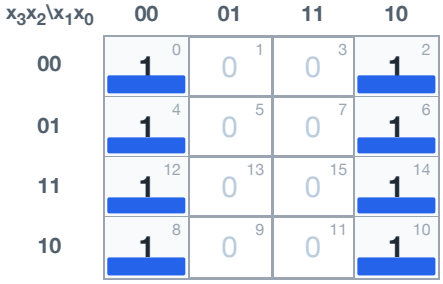
Y(1)



$Y(1) = x_1x_0 + \bar{x_1}\bar{x_0}$

$= x_1 \odot x_0$

Y(0)



$Y(0) = \bar{x_0}$

$= \bar{x}_0$

> **Intuicja**
>
> 3 = 0011. Przeniesienie do bitu 2: $c_2 = x_1 + x_0$; do bitu 3: $c_3 = x_2 \cdot c_2$.

Kod VHDL — `Z1_b.vhd`

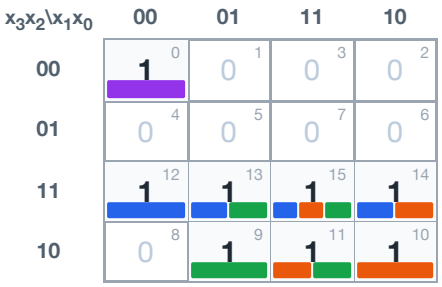
```
-- Z1 b) Y = (X + 3) mod 16
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Z1 is
    Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
           Y : out STD_LOGIC_VECTOR (3 downto 0));
end Z1;

architecture Dataflow of Z1 is
begin
    Y(0) <= not X(0);
    Y(1) <= (X(1) and X(0)) or (not X(1) and not X(0));
    Y(2) <= (X(2) and not X(1) and not X(0))
         or (not X(2) and X(1)) or (not X(2) and X(0));
    Y(3) <= (X(3) and not X(2)) or (X(3) and not X(1) and not X(0))
         or (not X(3) and X(2) and X(1)) or (not X(3) and X(2) and X(0));
end Dataflow;
```

Wariant c) Y = (X – 1) mod 16

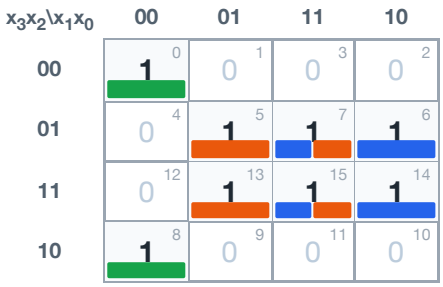
Y(3)



$Y(3) = x_3x_2 + x_3x_1 + x_3x_0 + \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0}$

$= x_3 \oplus (\bar{x}_2\bar{x}_1\bar{x}_0)$

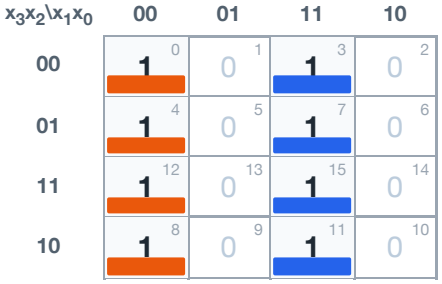
Y(2)



$Y(2) = x_2x_1 + x_2x_0 + \overline{x_2}\,\overline{x_1}\,\overline{x_0}$

$= x_2 \oplus (\bar{x}_1\bar{x}_0)$

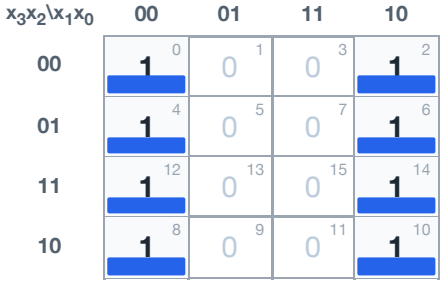
Y(1)



$Y(1) = x_1x_0 + \overline{x_1}\,\overline{x_0}$

$= x_1 \odot x_0$

Y(0)



$Y(0) = \overline{x_0}$

$= \bar{x}_0$

> **Intuicja**
>
> Bit i przełącza się, gdy wszystkie niższe bity = 0 (pożyczka). Lustro a): c(X) = NOT a(NOT X).

Kod VHDL — `Z1_c.vhd`

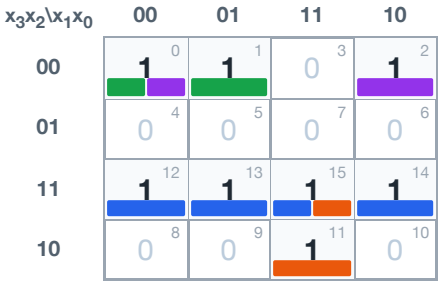
```
-- Z1 c) Y = (X - 1) mod 16
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Z1 is
    Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
           Y : out STD_LOGIC_VECTOR (3 downto 0));
end Z1;

architecture Dataflow of Z1 is
begin
    Y(0) <= not X(0);
    Y(1) <= (X(1) and X(0)) or (not X(1) and not X(0));
    Y(2) <= (X(2) and X(1)) or (X(2) and X(0))
         or (not X(2) and not X(1) and not X(0));
    Y(3) <= (X(3) and X(2)) or (X(3) and X(1)) or (X(3) and X(0))
         or (not X(3) and not X(2) and not X(1) and not X(0));
end Dataflow;
```

Wariant d) Y = (X – 3) mod 16

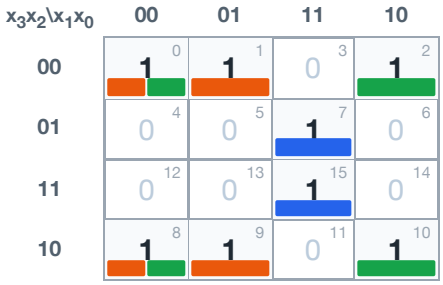
Y(3)



$Y(3) = x_3x_2 + x_3x_1x_0 + \bar{x}_3\bar{x}_2\bar{x}_1 + \bar{x}_3\bar{x}_2\bar{x}_0$

$= x_3 \oplus \bar{x}_2(\bar{x}_1 + \bar{x}_0)$

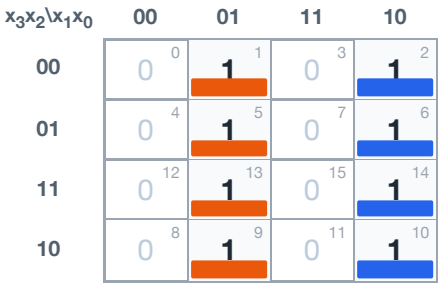
Y(2)



$Y(2) = x_2x_1x_0 + \bar{x}_2\bar{x}_1 + \bar{x}_2\bar{x}_0$

$= x_2 \odot (x_1x_0)$

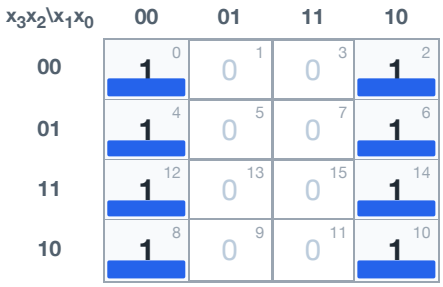
Y(1)



$Y(1) = x_1\bar{x}_0 + \bar{x}_1x_0$

$= x_1 \oplus x_0$

Y(0)



$Y(0) = \bar{x}_0$

$= \bar{x}_0$

> **Intuicja**
>
> Pożyczka do bitu 2: $b_2 = \bar{x}_1 + \bar{x}_0$; do bitu 3: $b_3 = \bar{x}_2 \cdot b_2$. Lustro b): d(X) = NOT b(NOT X).

Kod VHDL — `Z1_d.vhd`

```
-- Z1 d) Y = (X - 3) mod 16
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Z1 is
    Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
           Y : out STD_LOGIC_VECTOR (3 downto 0));
end Z1;

architecture Dataflow of Z1 is
begin
    Y(0) <= not X(0);
    Y(1) <= (X(1) and not X(0)) or (not X(1) and X(0));
    Y(2) <= (X(2) and X(1) and X(0))
         or (not X(2) and not X(1)) or (not X(2) and not X(0));
    Y(3) <= (X(3) and X(2)) or (X(3) and X(1) and X(0))
         or (not X(3) and not X(2) and not X(1)) or (not X(3) and not X(2) and not X(0));
end Dataflow;
```

Testbench i plik XDC

`Z1_tb.vhd`

```
-- Testbench Z1: X = 0..15, co 100 ns.
-- K = przesuniecie wariantu: a) 1, b) 3, c) -1, d) -3 (tylko do samosprawdzenia).
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Z1_tb is
    Generic ( K : integer := 1 );
end Z1_tb;

architecture Behavioral of Z1_tb is
    component Z1
        Port ( X : in  STD_LOGIC_VECTOR (3 downto 0);
               Y : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal X : STD_LOGIC_VECTOR (3 downto 0);
    signal Y : STD_LOGIC_VECTOR (3 downto 0);
begin
    UUT: Z1 port map ( X => X, Y => Y );

    X <= "0000",
         "0001" after  100 ns, "0010" after  200 ns, "0011" after  300 ns,
         "0100" after  400 ns, "0101" after  500 ns, "0110" after  600 ns,
         "0111" after  700 ns, "1000" after  800 ns, "1001" after  900 ns,
         "1010" after 1000 ns, "1011" after 1100 ns, "1100" after 1200 ns,
         "1101" after 1300 ns, "1110" after 1400 ns, "1111" after 1500 ns;

    -- Samosprawdzenie (opcjonalne, na zajeciach mozna pominac):
    -- w polowie kazdego kroku porownuje Y z (X + K) mod 16.
    Check: process
    begin
        for i in 0 to 15 loop
            wait for 50 ns;
            assert to_integer(unsigned(Y)) = (i + K) mod 16
                report "Blad dla X = " & integer'image(i) severity error;
            wait for 50 ns;
        end loop;
        report "Koniec testu";
        wait;
    end process;
end Behavioral;
```

> **Generic K**
>
> Ustaw na swój wariant: a) `1`, b) `3`, c) `-1`, d) `-3`. Proces `Check` jest opcjonalny — prowadzący wymaga tylko przypisania `X <= …`.

`Z1.xdc`

```
## Z1: X(3:0) <- PL_USER_SW(3:0), Y(3:0) -> PL_USER_LED(3:0)
## Piny z master XDC AUP-ZU3 (Real Digital); SW zgodne z Vivado_Instrukcja.pdf 4.1.
## Na zajeciach i tak sprawdz z szablonem ZU3.xdc ze strony przedmiotu.

set_property PACKAGE_PIN AB1 [get_ports {X[0]}]
set_property PACKAGE_PIN AF1 [get_ports {X[1]}]
set_property PACKAGE_PIN AE3 [get_ports {X[2]}]
set_property PACKAGE_PIN AC2 [get_ports {X[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports X*]

set_property PACKAGE_PIN AF5 [get_ports {Y[0]}]
set_property PACKAGE_PIN AE7 [get_ports {Y[1]}]
set_property PACKAGE_PIN AH2 [get_ports {Y[2]}]
set_property PACKAGE_PIN AE5 [get_ports {Y[3]}]
set_property IOSTANDARD LVCMOS12 [get_ports Y*]
```

W domu na Macu (bez Vivado)

Vivado nie działa na macOS. Logikę i testbench sprawdzisz symulatorem **nvc** (już zainstalowany):

```
cd Z1
nvc -a Z1_a.vhd Z1_tb.vhd -e -gK=1  Z1_tb -r --wave=a.fst   # wariant a
nvc -a Z1_b.vhd Z1_tb.vhd -e -gK=3  Z1_tb -r                # wariant b
nvc -a Z1_c.vhd Z1_tb.vhd -e -gK=-1 Z1_tb -r                # wariant c
nvc -a Z1_d.vhd Z1_tb.vhd -e -gK=-3 Z1_tb -r                # wariant d
```

Sukces = tylko komunikat `Koniec testu`, bez `Blad dla X = …`. Przebiegi (`a.fst`) otworzysz w GTKWave lub Surfer.

Vivado krok po kroku

1. **Create Project** → RTL Project → płyta **AUP-ZU3 8GB**. W podsumowaniu sprawdź: Target language = **VHDL**.
2. **Add Sources** → **design source** → nowy plik `Z1`, porty: `X` in, bus 3:0; `Y` out, bus 3:0. Wpisz architekturę ze swojego wariantu.
3. **Add Sources** → **simulation source** → `Z1_tb` (bez portów), wklej testbench. W oknie Sources: `Z1_tb` jest top w Simulation Sources, pod nim `UUT : Z1` (ew. *Set as top*).
4. **Run Behavioral Simulation** (Flow Navigator → SIMULATION).
5. **Constraints**: dodaj `Z1.xdc` lub szablon `ZU3.xdc` — odkomentuj PL_USER_SW[0..3] i PL_USER_LED[0..3], zmień nazwy na `X` / `Y`, **zostaw linie IOSTANDARD**.
6. **Generate Bitstream** (synteza i implementacja uruchomią się same).
7. **Open Hardware Manager** → Open Target → Auto Connect → **Program Device** → plik `.bit` z `<projekt>.runs\impl_1` (sprawdź datę!).
8. Test na płycie: przełączniki SW3..SW0 = X, diody LED3..LED0 = Y. Np. a) SW = 0101 → LED = 0110.
9. **Run Simulation** → **Run Post-Implementation Timing Simulation** (po implementacji).

⚠ **Pułapki**

- Symulacja domyślnie trwa **1000 ns** → widać tylko X = 0…9. W Tcl Console: `run 600 ns` (albo Restart + `run 1600 ns`).
- Po zmianie kodu: **Relaunch** zamiast zamykania symulacji.
- `A and B or C` = **błąd składni** — mieszanie and/or wymaga nawiasów: `(A and B) or C`.
- Indeksy: VHDL `X(0)`, XDC `{X[0]}`.
- Błędy `UCIO-1` / `NSTD-1` przy bitstreamie = brak `PACKAGE_PIN` lub `IOSTANDARD` w XDC.
- Stary `.bit` z innego projektu — okno wyboru pliku pamięta ostatni folder.

Wygodna analiza przebiegów

- PPM na `X`, `Y` → **Radix** → **Unsigned Decimal** — od razu liczby 0–15.
- **Zoom Fit** — cały przebieg; **Next/Previous Transition** — skok kursora do zmiany sygnału.
- **Add Marker** — pomiar odstępów czasu (klik w marker = nowe „zero” skali).

Symulacja czasowa: X: 0 → 1

Zmienia się tylko X(0), ale na wyjściu przełączają się wszystkie bity, którymi różnią się f(0) i f(1):

Wariant	Y przed → po	Bity zmieniające się	Możliwe stany przejściowe
a)	`0001 → 0010` (1→2)	Y0 ↓, Y1 ↑	`0000 (0), 0011 (3)`
b)	`0011 → 0100` (3→4)	Y0 ↓, Y1 ↓, Y2 ↑	`0000 (0), 0001 (1), 0010 (2), 0101 (5), 0110 (6), 0111 (7)`
c)	`1111 → 0000` (15→0)	Y0 ↓, Y1 ↓, Y2 ↓, Y3 ↓	`0001 (1), 0010 (2), 0011 (3), 0100 (4), 0101 (5), 0110 (6), 0111 (7), 1000 (8), 1001 (9), 1010 (10), 1011 (11), 1100 (12), 1101 (13), 1110 (14)`
d)	`1101 → 1110` (13→14)	Y0 ↓, Y1 ↑	`1100 (12), 1111 (15)`

1. Opóźnienie propagacji

Y zmienia się dopiero kilka ns po zmianie X. Pomiar: kursor na zboczu X → *Add Marker* → zaznacz bit Y → *Next Transition*. Na opóźnienie składa się: bufor wejściowy (IBUF) → połączenia w matrycy → LUT → bufor wyjściowy (OBUF) → pin. Największa część to bufory I/O.

2. Rozrzut (skew) i hazardy

Każdy bit Y ma własną ścieżkę (inna LUT, inny routing, inny pin), więc bity przełączają się w nieco różnych chwilach. Na magistrali Y (w trybie dziesiętnym) pojawia się na moment **fałszywa wartość** — glitch / hazard. Im więcej bitów zmienia się jednocześnie, tym więcej możliwych stanów pośrednich — najgorszy jest wariant c) (wszystkie 4 bity). Bity, które się nie zmieniają, pozostają stabilne (każdy to osobna LUT, a zmienia się tylko jedno jej wejście).

3. Wniosek

Wyjście układu kombinacyjnego jest poprawne dopiero po ustaleniu się wszystkich bitów. Dlatego w układach synchronicznych wynik próbkuje się zboczem zegara — okres zegara musi być dłuższy niż najdłuższa ścieżka (ścieżka krytyczna).

Uwaga: GSR

W symulacji czasowej przez pierwsze ~100 ns aktywny jest globalny reset (GSR). Zmiana X 0→1 następuje dokładnie w 100 ns — jeśli to przeszkadza, wydłuż pierwszy krok: `"0001" after 200 ns`.